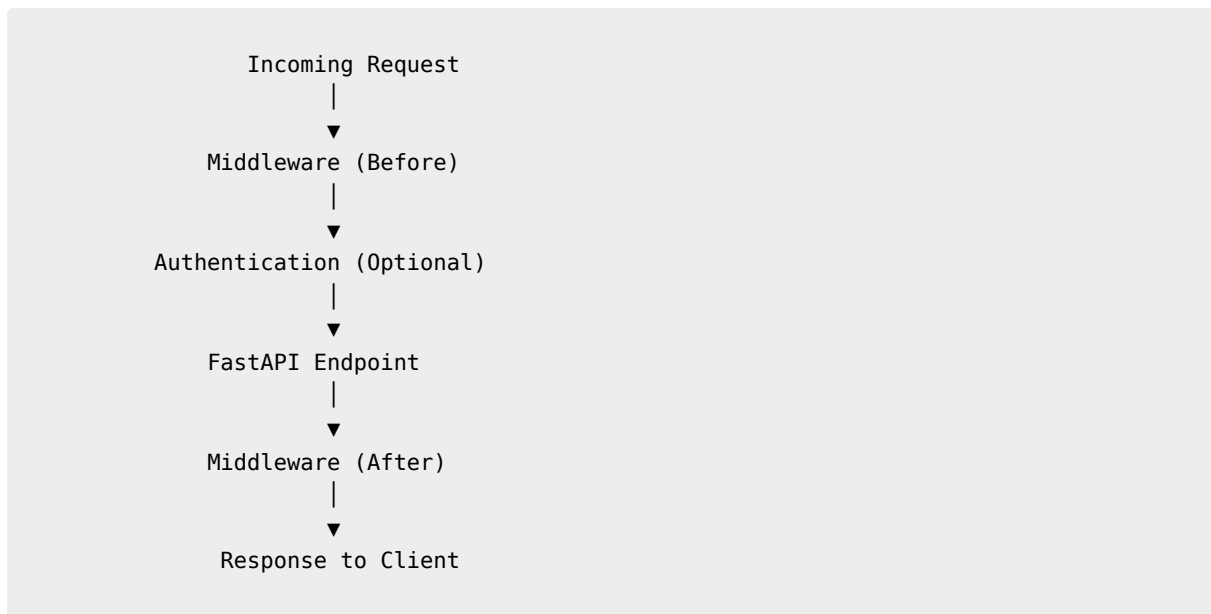


What is Middleware?

Middleware is code that executes:

1. **Before** the request reaches your API endpoint.
2. **After** the endpoint finishes and before the response is sent back.

The request/response lifecycle looks like this:



Unlike dependencies, middleware runs **for every request** unless you explicitly exclude routes.

Why do we need Middleware?

Imagine you have 100 APIs:

```
GET /users
POST /users
GET /products
POST /orders
DELETE /employee
PUT /department
...
```

Suppose you want to log every request.

Without middleware, you'd write this in every endpoint:

```
@app.get("/users")
async def get_users():
    print("Request received")
    return {"message": "Users"}
```

```
@app.post("/products")
async def create_product():
    print("Request received")
    return {"message": "Product Created"}
```

This repeats the same code many times.

Instead, use middleware:

```
@app.middleware("http")
async def log_request(request, call_next):
    print("Request received")
    response=await call_next(request)
    return response
```

Now **every** endpoint is logged automatically. FastAPI's middleware wraps the entire request/response cycle and is intended for cross-cutting concerns such as logging, timing, and adding headers.

How Middleware Works

Suppose you call:

```
GET /users
```

Execution order:

```
Client
|
▼
Middleware starts
|
▼
Endpoint executes
|
▼
```

```
Endpoint returns response
|
▼
Middleware modifies response
|
▼
Client receives response
```

The Middleware Function

```
@app.middleware("http")
async def my_middleware(request, call_next):

    # Before endpoint
    print("Before")

    response=await call_next(request)

    # After endpoint
    print("After")

    return response
```

Let's explain each line.

@app.middleware("http")

Registers this function as HTTP middleware.

```
@app.middleware("http")
```

Meaning:

| "Execute this function for every HTTP request."

request

```
async def my_middleware(request, call_next):
```

request contains everything sent by the client.

For example:

```
GET /users?id=5
```

The request contains:

- URL
- Method
- Headers
- Cookies
- Query parameters
- Client IP

Example:

```
print(request.url)
```

Output

```
http://localhost:8000/users
```

```
print(request.method)
```

Output

```
GET
```

```
print(request.headers)
```

Output

```
Host: localhost:8000
```

User-Agent: Postman

call_next

This is the most important line.

```
response=await call_next(request)
```

Think of `call_next()` as:

└ "Continue processing the request."

Without it:

```
@app.middleware("http")
async def middleware(request, call_next):
    print("Before")
    return {"message": "Hello"}
```

The request **never reaches the endpoint**.

Flow:

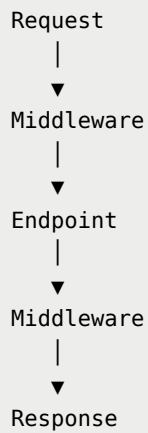
```
Request
|
▼
Middleware
|
▼
Return Response
```

Endpoint is skipped.

With `call_next()`:

```
response=await call_next(request)
```

Flow:



Example

Endpoint

```
@app.get("/hello")
async def hello():

    print("Inside endpoint")

    return {"msg":"Hello"}
```

Middleware

```
@app.middleware("http")
async def middleware(request,call_next):
    print("Before endpoint")
    response=await call_next(request)
    print("After endpoint")

    return response
```

Console

```
Before endpoint

Inside endpoint

After endpoint
```

This demonstrates the order in which middleware surrounds your endpoint execution.

Real Example 1 — Logging

```
@app.middleware("http")
async def logging(request, call_next):

    print(f"Request :{request.method}")
    print(f"URL :{request.url}")
    response=await call_next(request)
    print(f"Status :{response.status_code}")

    return response
```

Output

```
Request : GET

URL : http://localhost:8000/users

Status : 200
```

Real Example 2 — Measure API Time

```
import time

@app.middleware("http")
async def timer(request, call_next):

    start=time.time()
    response=await call_next(request)
    end=time.time()
    print(end-start)

    return response
```

Output

```
0.0421 seconds
```

Useful for monitoring slow APIs.

Real Example 3 — Add Custom Header

```
@app.middleware("http")
async def header(request, call_next):

    response=await call_next(request)

    response.headers["App"]="FastAPI"

    return response
```

Every response now contains:

```
App : FastAPI
```

Real Example 4 — Block Requests

```
from fastapi.responses import JSONResponse

@app.middleware("http")
async def block(request, call_next):

    if request.method=="DELETE":
        return JSONResponse(
            {"message": "DELETE blocked"},
            status_code=403
        )

    return await call_next(request)
```

Flow:

```
DELETE /users

↓

Middleware

↓
```


403 Forbidden

Endpoint is never executed.

Middleware vs Dependencies

Suppose only `/users` needs authentication.

Use a dependency:

```
@app.get("/users")
async def users(
    current_user=Depends(get_current_user)
):
    ...
```

Only `/users` is affected.

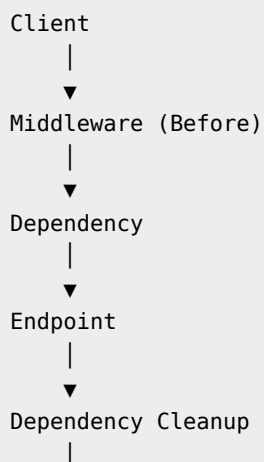
Suppose **every request** should be logged.

Use middleware:

```
@app.middleware("http")
```

Now all endpoints are logged.

Request Lifecycle





Dependencies are ideal for route-specific concerns such as database sessions or authentication, while middleware is intended for global behavior that should apply to all requests.

Common Production Uses of Middleware

Middleware	Why it's used
Logging	Record every incoming request and response status.
Request timing	Measure API performance and identify slow endpoints.
CORS	Allow browsers from approved origins to access your API.
Security headers	Add headers like <code>X-Frame-Options</code> or <code>X-Content-Type-Options</code> .
Request ID	Attach a unique ID to each request for tracing across logs.
Rate limiting (often with third-party middleware)	Restrict how many requests a client can make.
Compression	Compress responses (e.g., GZip) to reduce bandwidth.
Global exception handling	Format unhandled errors consistently before sending them to clients.

Interview Answer

Question: Why do we use middleware?

Answer:

Middleware is used for logic that must run for **every HTTP request and response**, regardless of which endpoint is called. It executes before the request reaches the endpoint and again after the endpoint returns a response.

Typical uses include logging, request timing, adding security or custom headers, CORS handling, compression, and assigning request IDs. It keeps these cross-cutting concerns centralized instead of duplicating code in every API endpoint.